

Big Data – TP2 : Spark DataFrame & SQL

Bastien CETRE – Ahmed EL AYEB 3A SDI 9 octobre 2026

On a travaillé comme au TP1 sur le cluster du DCE (HDFS et Spark sur `sar17`, comptes `sdim_9` et `sdim_14`), depuis un nœud `kyle` obtenu avec `srun`. Dans les templates, on a remplacé `sarXX` par `sar17` et le dossier de sortie par `/sdim/sdim_9/`. Les temps sont ceux affichés par la fonction `time_usage` des scripts.

1 Nombre de partitions d'un RDD

1.1 Mesures

Le script `avg_temperatures_rdd.py` reprend la version rapide du TP1 (paires (`année`, (`t`, 1)) puis `reduceByKey`). On a juste mis les bons chemins :

```
input_path = "hdfs://sar17:9000/data/temperatures/"
output_path = "hdfs://sar17:9000/sdim/sdim_9/"
```

On l'a lancé sur les quatre fichiers (pour pouvoir comparer les dossiers de sortie en 1.2) :

```
$ for f in 86400 2880 86 10; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_${f}.rdd.out; echo "== temperatures_${f}";
spark-submit --master spark://sar17:7077 avg_temperatures_rdd.py temperatures_${f}.csv 2>&1 | grep -E "partitions|elapsed"; done
== temperatures_86400
Number of partitions of the input RDD: 2
Number of partitions of the output RDD: 2
INFO:root:Save as text file execution time: elapsed seconds: 3.292
== temperatures_2880
Number of partitions of the input RDD: 2
Number of partitions of the output RDD: 2
INFO:root:Save as text file execution time: elapsed seconds: 3.5105
== temperatures_86
Number of partitions of the input RDD: 2
Number of partitions of the output RDD: 2
INFO:root:Save as text file execution time: elapsed seconds: 5.4414
== temperatures_10
Number of partitions of the input RDD: 24
Number of partitions of the output RDD: 24
INFO:root:Save as text file execution time: elapsed seconds: 22.4356
```

Fichier	Temps (s)	Partitions <code>text_file</code>	Partitions <code>temperatures</code>
<code>temperatures_86400.csv</code>	3,29	2	2
<code>temperatures_2880.csv</code>	3,51	2	2
<code>temperatures_86.csv</code>	5,44	2	2
<code>temperatures_10.csv</code>	22,44	24	24

Tableau 1 – RDD : temps et nombre de partitions.

1.2 Analyse

Partitions en entrée.

```
$ hdfs dfs -ls hdfs://sar17:9000/data/temperatures/temperatures_10.csv
-rw-r--r-- 3 cpu_vialle cpu_prof 3180285744 2026-08-17 10:00 hdfs://sar17:9000/data/temperatures/temperatures_10.csv
```

Le fichier `temperatures_10.csv` fait 3 180 285 744 octets. Divisé par 24, ça donne environ 132 Mo par partition, c'est-à-dire à peu près un bloc HDFS de 128 Mio (134 217 728 octets). Le fichier fait 23,7 blocs, donc 24 blocs, et Spark crée une partition par bloc. Les trois petits fichiers tiennent dans un seul bloc mais ont quand même 2 partitions : `textFile` en crée au moins 2 par défaut.

Fichiers de sortie.

```
$ for f in 86400 2880 86 10; do echo "== temperatures_${f}"; hdfs dfs -ls hdfs://sar17:9000/sdim/sdim_9/temperatures_${f}.rdd.out; done
== temperatures_86400
Found 3 items
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86400.rdd.out/_SUCCESS
-rw-r--r-- 3 sdim_9 sdim 445 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86400.rdd.out/part-00000
-rw-r--r-- 3 sdim_9 sdim 645 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86400.rdd.out/part-00001
== temperatures_2880
Found 3 items
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_2880.rdd.out/_SUCCESS
-rw-r--r-- 3 sdim_9 sdim 448 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_2880.rdd.out/part-00000
-rw-r--r-- 3 sdim_9 sdim 644 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_2880.rdd.out/part-00001
== temperatures_86
Found 3 items
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86.rdd.out/_SUCCESS
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86.rdd.out/part-00000
-rw-r--r-- 3 sdim_9 sdim 29 2026-10-09 13:59 hdfs://sar17:9000/sdim/sdim_9/temperatures_86.rdd.out/part-00001
== temperatures_10
Found 25 items
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/_SUCCESS
-rw-r--r-- 3 sdim_9 sdim 28 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00000
-rw-r--r-- 3 sdim_9 sdim 29 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00001
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00002
-rw-r--r-- 3 sdim_9 sdim 112 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00003
```

```

-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00004
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00005
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00006
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00007
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00008
-rw-r--r-- 3 sdim_9 sdim 55 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00009
-rw-r--r-- 3 sdim_9 sdim 29 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00010
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00011
-rw-r--r-- 3 sdim_9 sdim 28 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00012
-rw-r--r-- 3 sdim_9 sdim 28 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00013
-rw-r--r-- 3 sdim_9 sdim 85 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00014
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00015
-rw-r--r-- 3 sdim_9 sdim 55 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00016
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00017
-rw-r--r-- 3 sdim_9 sdim 28 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00018
-rw-r--r-- 3 sdim_9 sdim 112 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00019
-rw-r--r-- 3 sdim_9 sdim 84 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00020
-rw-r--r-- 3 sdim_9 sdim 28 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00021
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00022
-rw-r--r-- 3 sdim_9 sdim 56 2026-10-09 14:00 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.rdd.out/part-00023

```

Il y a un fichier `part-*` par partition de `temperatures` (2 ou 24), plus le fichier vide `_SUCCESS`. Spark ne rassemble donc pas le résultat avant de l'écrire : chaque tâche écrit sa partition dans son propre fichier. Le `reduceByKey` garde le même nombre de partitions que l'entrée et répartit les années par hachage. Comme on n'a que 39 années (une ligne d'environ 28 octets chacune) pour 24 partitions, certaines en reçoivent 2 ou 4 et d'autres aucune, d'où les fichiers vides. Pour `temperatures_86`, il n'y a qu'une année, qui tombe dans la partition 1.

2 Calcul des moyennes avec l'API DataFrame

2.1 Code et mesures

On lit le CSV en donnant le schéma, pour que Spark n'ait pas à le deviner, puis on fait un `groupBy` sur l'année. On a ajouté un tri pour avoir une sortie dans l'ordre.

```

def avg_temperature_df(df):
    return df.groupBy("year") \
        .agg(avg("temperature").alias("avg_temperature")) \
        .orderBy("year")

schema = StructType([StructField("year", IntegerType(), True),
    StructField("month", IntegerType(), True), StructField("day", IntegerType(), True),
    StructField("hour", IntegerType(), True), StructField("minute", IntegerType(), True),
    StructField("second", IntegerType(), True), StructField("temperature", DoubleType(), True)])
df = spark.read.csv(input_file, schema=schema, header=False)
df_avg = avg_temperature_df(df)

```

Pour avoir le nombre de partitions, on affiche `df_avg.rdd.getNumPartitions()`. Au début on l'avait mis avant l'écriture, et on trouvait 10 s pour le gros fichier. Mais avec le `orderBy`, cet appel lance déjà le calcul (Spark doit regarder les données pour savoir comment découper le tri), et le `write` chronométré profitait de ce travail. On l'a donc déplacé après l'écriture :

```

$ for f in 86400 2880 86 10; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_${f}.df.out*; echo "== temperatures_${f}";
    spark-submit --master spark://sar17:7077 avg_temperatures_df.py temperatures_${f}.csv 2>&1 | grep -E "partitions|elapsed|Error"; done
== temperatures_86400
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 5.7308
Number of partitions of df_avg: 1
== temperatures_2880
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 5.7422
Number of partitions of df_avg: 1
== temperatures_86
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 5.7348
Number of partitions of df_avg: 1
== temperatures_10
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 14.0507
Number of partitions of df_avg: 1
$ hdfs dfs -cat hdfs://sar17:9000/sdim/sdim_9/temperatures_86400.df.out/* | tail -3
2016,8.337049180327872
2017,7.345287671232872
2018,8.23512328767123
$ hdfs dfs -ls hdfs://sar17:9000/sdim/sdim_9/temperatures_10.df.out
Found 2 items
-rw-r--r-- 3 sdim_9 sdim 0 2026-10-09 14:26 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.df.out/_SUCCESS
-rw-r--r-- 3 sdim_9 sdim 898 2026-10-09 14:26 hdfs://sar17:9000/sdim/sdim_9/temperatures_10.df.out/part-00000-d5c46b15-778e-4ffc
    -8060-bbb8b5d5f4d1-c000.csv

```

Fichier	RDD (s)	DataFrame (s)	Partitions df_avg
temperatures_86400.csv	3,29	5,73	1
temperatures_2880.csv	3,51	5,74	1
temperatures_86.csv	5,44	5,73	1
temperatures_10.csv	22,44	14,05	1

Tableau 2 – RDD et DataFrame.

Les moyennes sont les mêmes qu'avec les RDD. Le résultat n'a qu'une partition, donc un seul fichier `part-*` (898 octets) au lieu de 24 : Spark SQL regroupe les partitions après le shuffle quand il y a peu de données, et 39 lignes tiennent dans une seule.

2.2 Comparaison avec les RDD

Sur les petits fichiers, les DataFrames sont plus lents (5,7 s, quelle que soit la taille). Ce temps ne dépend pas des données : c'est le prix de Spark SQL, qui doit analyser et optimiser la requête (Catalyst), générer le code et lancer plusieurs jobs (agrégation puis tri). Sur quelques Mo, ça coûte plus que le calcul lui-même.

Sur le fichier de 3 Go, c'est l'inverse : 14 s au lieu de 22 s. Avec les RDD, chaque ligne part dans un processus Python pour le `split`, le `float` et la création du tuple, puis revient dans la JVM. Avec les DataFrames, tout reste dans la JVM : lecture du CSV, calcul et agrégation sont faits par du code compilé, sur des données en format binaire. Spark sait aussi qu'il n'a besoin que de deux colonnes. Le coût fixe est vite rattrapé quand les données grossissent.

2.3 Mise en cache

2.3.1 Sans cache. On décommente la deuxième écriture (`output_folder+".bis"`) :

```
$ hdfs dfs -rm -r -f hdfs://sari17:9000/sdim/sdim_9/temperatures_10.df.out*; spark-submit --master spark://sari17:7077 avg_temperatures_df.py temperatures_10.csv 2>&1 | grep -E "partitions|elapsed|Error"
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 14.1672
Number of partitions of df_avg: 1
INFO:root:Execution time of second action df_avg.write.csv: elapsed seconds: 9.1088
```

Les deux écritures prennent du temps, parce que Spark est paresseux : `read.csv` et `groupBy` ne calculent rien, ils construisent seulement un plan. Le calcul n'a lieu qu'au moment d'une action comme `write`, et le résultat n'est pas gardé ensuite. La deuxième écriture relit donc les 3 Go et refait tout. Elle est un peu plus rapide parce que la première a payé seule les coûts de démarrage (lancement des executors, chargement de la JVM) et que le fichier est sans doute encore dans le cache disque des machines, mais 9 s, c'est bien une relecture complète.

2.3.2 Avec cache. On ajoute `df_avg.cache()` juste après avoir défini `df_avg`, donc avant la première action :

```
$ hdfs dfs -rm -r -f hdfs://sari17:9000/sdim/sdim_9/temperatures_10.df.out*; spark-submit --master spark://sari17:7077 avg_temperatures_df.py temperatures_10.csv 2>&1 | grep -E "partitions|elapsed|Error"
INFO:root:Execution time of first action df_avg.write.csv: elapsed seconds: 18.1657
Number of partitions of df_avg: 39
INFO:root:Execution time of second action df_avg.write.csv: elapsed seconds: 3.7408
```

Cette fois la deuxième écriture ne prend que 3,7 s : elle lit les 39 lignes gardées en mémoire au lieu de recalculer. Ce qui reste, c'est le lancement du job et l'écriture dans HDFS. La première écriture est un peu plus longue (18 s) car elle remplit aussi le cache. On remarque aussi 39 partitions au lieu d'une : quand un DataFrame est en cache, Spark ne regroupe plus les partitions après le shuffle, et le tri par année en donne une par année.

`cache()` ne calcule rien tout seul : il dit juste à Spark de garder le résultat la prochaine fois qu'il sera calculé. C'est pour ça qu'il faut le placer avant la première action. On retrouve le modèle d'exécution de Spark : les transformations construisent un graphe (DAG), qui n'est exécuté qu'à l'appel d'une action, et entièrement refait à chaque action sauf ce qui a été mis en cache.

3 Calcul des moyennes en SQL

3.1 Avec une vue

On crée une vue temporaire et on écrit la même requête qu'en 2.1 en SQL :

```
def avg_temperature_sql(df, view_name):
    df.createOrReplaceTempView(view_name)
    return spark.sql("""
        SELECT year, AVG(temperature) AS avg_temperature
        FROM {} GROUP BY year ORDER BY year
        """.format(view_name))

schema = "year INT, month INT, day INT, hour INT, minute INT, second INT, temperature DOUBLE"
df = spark.read.csv(input_file, schema=schema, header=False)
```

```
$ for f in 86400 2880 86 10; do hdfs dfs -rm -r -f hdfs://sari17:9000/sdim/sdim_9/temperatures_${f}.sql.out; echo "== temperatures_${f}";
spark-submit --master spark://sari17:7077 avg_temperatures_sql_view.py temperatures_${f}.csv 2>&1 | grep -E "elapsed|Error"; done
== temperatures_86400
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 5.133
== temperatures_2880
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 5.376
== temperatures_86
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 5.7008
== temperatures_10
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 14.0374
2016,8.337049180327872
2017,7.345287671232872
2018,8.23512328767123
```

On obtient quasiment les mêmes temps qu'avec l'API DataFrame (14,04 s contre 14,05 s sur 3 Go). Ce n'est pas étonnant : une vue ne contient aucune donnée, c'est juste un nom donné au DataFrame. La requête SQL et le `groupBy().agg()` donnent le même plan une fois passés par Catalyst. Choisir SQL ou l'API DataFrame est donc une question de goût, pas de performance.

3.2 Avec une table

On a pris `avg_temperatures_sql_table.py`. On crée d'abord le dossier `spark-warehouse` (`hdfs dfs -mkdir`), puis on complète les chemins, la lecture et la même requête SQL, cette fois sur la table :

```
metastore_location = "hdfs://sar17:9000/sdim/sdim_9/spark-warehouse"
...
return spark.sql(f"""
    SELECT year, AVG(temperature) AS avg_temperature
    FROM {table_name} GROUP BY year ORDER BY year
""")
```

Le template écrit la table avec `saveAsTable` seulement si son dossier n'existe pas encore dans `spark-warehouse`. Sinon, il la redéclare à partir des fichiers déjà écrits (`CREATE TABLE ... USING PARQUET LOCATION ...`).

3.2.1 Première exécution (dossier spark-warehouse vide au départ) :

```
$ for f in 86400 2880 86 10; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_${f}.sql.table.out; echo "== temperatures_${f}"
"; spark-submit --master spark://sar17:7077 avg_temperatures_sql_table.py temperatures_${f}.csv 2>&1 | grep -E "elapsed|Error"; done
== temperatures_86400
INFO:root:Execution needed to write the table: : elapsed seconds: 4.6677
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 4.814
== temperatures_2880
INFO:root:Execution needed to write the table: : elapsed seconds: 4.7516
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 5.1385
== temperatures_86
INFO:root:Execution needed to write the table: : elapsed seconds: 5.1394
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 5.1548
== temperatures_10
INFO:root:Execution needed to write the table: : elapsed seconds: 21.5835
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 3.8776
```

Fichier	RDD (s)	DataFrame (s)	Vue SQL (s)	Table SQL (s) écriture table	requête
temperatures_86400.csv	3,29	5,73	5,13	4,67	4,81
temperatures_2880.csv	3,51	5,74	5,38	4,75	5,14
temperatures_86.csv	5,44	5,73	5,70	5,14	5,15
temperatures_10.csv	22,44	14,05	14,04	21,58	3,88

Tableaux 3 et 4 – toutes les versions.

Pour les petits fichiers, rien ne change : environ 5 s, toujours le coût fixe de Spark SQL. Pour le gros fichier, la requête sur la table ne prend que 3,9 s, contre 14 s avec la vue. Mais il a fallu écrire la table avant, ce qui a pris 21,6 s, donc au total la première exécution est plus longue qu'avec une vue.

La différence vient du format. `saveAsTable` écrit en Parquet : un format binaire, compressé et rangé par colonnes. Pour la moyenne, Spark ne lit que les colonnes `year` et `temperature`, déjà converties, au lieu de relire 3 Go de texte. La table est d'ailleurs 16 fois plus petite que le CSV (196,6 Mo, 589,9 Mo avec les 3 copies HDFS) :

```
$ hdfs dfs -du -s -h hdfs://sar17:9000/sdim/sdim_9/spark-warehouse/temperatures_10
196.6 M 589.9 M hdfs://sar17:9000/sdim/sdim_9/spark-warehouse/temperatures_10
```

3.2.3 Deuxième exécution. On supprime seulement le dossier de sortie et on relance :

```
$ hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_10.sql.table.out; spark-submit --master spark://sar17:7077
avg_temperatures_sql_table.py temperatures_10.csv 2>&1 | grep -E "elapsed|Error"
INFO:root:Execution time of the action df_avg.write.csv: elapsed seconds: 6.291
```

Il n'y a plus de ligne « write the table ». Le code vérifie avec `hdfs dfs -test -d` si le dossier de la table existe. C'est le cas, donc il ne relit pas le CSV : il redéclare la table à partir des fichiers Parquet de la première exécution et lance directement la requête. On passe de 25 s à 6,3 s, soit plus de deux fois mieux que la vue. La requête est un peu plus lente que les 3,9 s d'avant car c'est maintenant la première action du programme, qui paie le démarrage. Une table vaut donc le coup dès qu'on interroge plusieurs fois les mêmes données.

4 API DataFrame sur de gros fichiers

4.1 Les requêtes

On a gardé la lecture du template (schéma deviné sur 1 % des lignes) et écrit les 5 requêtes avec l'API DataFrame :

```
from pyspark.sql.functions import max as max_, sum as sum_, col

q1_result = df_customer.count()
q2_result = df_store_sales.agg(max_("ss_list_price").alias("max_list_price"))
```

```

q3_result = df_store_sales.groupBy("ss_customer_sk") \
    .agg(sum_("ss_net_paid_inc_tax").alias("amountSpent"))
q4_result = df_store_sales.groupBy("ss_customer_sk") \
    .agg(sum_("ss_net_paid_inc_tax").alias("amountSpent")) \
    .orderBy(col("amountSpent").desc())
q5_result = df_store_sales.join(df_customer,
    df_store_sales.ss_customer_sk == df_customer.c_customer_sk) \
    .groupBy("ss_customer_sk", "c_first_name", "c_last_name") \
    .agg(sum_("ss_net_paid_inc_tax").alias("amountSpent")) \
    .orderBy(col("amountSpent").desc()) \
    .select("c_first_name", "c_last_name", "amountSpent")

```

Q1 renvoie directement un nombre car `count()` est une action ; les autres renvoient un `DataFrame`, calculé au moment du `show`. Pour Q5, on met aussi le prénom et le nom dans le `groupBy` pour pouvoir les afficher (chaque client n'en a qu'un, donc le résultat ne change pas). Test sans cache sur le plus petit fichier :

```

$ spark-submit --master spark://sar17:7077 dataframe_api_benchmark.py store_sales_1_4.100.dat 2>&1 \
  | grep -E "elapsed|Number of customers|Error|\\| *"[0-9]" | head -60
INFO:root:Read input files : elapsed seconds: 13.231
Number of customers: 65000000
INFO:root:Query q1 (iteration 1): elapsed seconds: 2.8163
|      200.0|
INFO:root:Query q2 (iteration 1): elapsed seconds: 7.5088
|      355419|      83367.97|
INFO:root:Query q3 (iteration 1): elapsed seconds: 9.4709
|      NULL|2.908589506669999E9|
INFO:root:Query q4 (iteration 1): elapsed seconds: 9.1665
|      Michael|      Thorne|339502.50000000006|
INFO:root:Query 5 (iteration 1): elapsed seconds: 24.8821
Number of customers: 65000000
INFO:root:Query q1 (iteration 2): elapsed seconds: 2.0694
|      200.0|
INFO:root:Query q2 (iteration 2): elapsed seconds: 7.1785
|      859308|94002.31999999999|
INFO:root:Query q3 (iteration 2): elapsed seconds: 8.6737
|      NULL|2.90858950667E9|
INFO:root:Query q4 (iteration 2): elapsed seconds: 8.6805
|      Michael|      Thorne|339502.49999999994|
INFO:root:Query 5 (iteration 2): elapsed seconds: 23.5771
Number of customers: 65000000
INFO:root:Query q1 (iteration 3): elapsed seconds: 2.0193
|      200.0|
INFO:root:Query q2 (iteration 3): elapsed seconds: 7.1047
|      355419|      83367.97|
INFO:root:Query q3 (iteration 3): elapsed seconds: 8.5281
|      NULL|2.90858950667E9|
INFO:root:Query q4 (iteration 3): elapsed seconds: 8.4697
|      Michael|      Thorne|339502.5000000001|
INFO:root:Query 5 (iteration 3): elapsed seconds: 23.1977
Number of customers: 65000000
INFO:root:Query q1 (iteration 4): elapsed seconds: 1.9415
|      200.0|
INFO:root:Query q2 (iteration 4): elapsed seconds: 7.6327
|      276028|98076.79999999999|
INFO:root:Query q3 (iteration 4): elapsed seconds: 9.1266
|      NULL|2.9085895066700006E9|
INFO:root:Query q4 (iteration 4): elapsed seconds: 8.9421
|      Michael|      Thorne|339502.50000000006|
INFO:root:Query 5 (iteration 4): elapsed seconds: 24.4706
Number of customers: 65000000
INFO:root:Query q1 (iteration 5): elapsed seconds: 1.9426
|      200.0|
INFO:root:Query q2 (iteration 5): elapsed seconds: 7.1259
|      310728|      42413.97|
INFO:root:Query q3 (iteration 5): elapsed seconds: 9.2112
|      NULL|2.908589506669999E9|
INFO:root:Query q4 (iteration 5): elapsed seconds: 9.0099
|      Michael|      Thorne|339502.49999999994|
INFO:root:Query 5 (iteration 5): elapsed seconds: 23.5395

```

65 millions de clients, un prix maximum de 200 : ça semble cohérent. En tête de Q4 on trouve la clé `NULL`, qui regroupe les ventes sans client ; elles disparaissent avec la jointure de Q5. Le client affiché par Q3 change à chaque itération, ce qui est normal puisque le résultat n'est pas trié. Les derniers chiffres des sommes bougent aussi un peu, car les nombres à virgule ne sont pas additionnés dans le même ordre d'une fois sur l'autre.

4.2 Temps d'exécution

On décommente les deux `cache()` et on lance les quatre fichiers en arrière-plan (`nohup`), avec un log par fichier :

```

$ nohup bash -c 'for s in 100 200 400 800; do spark-submit --master spark://sar17:7077 dataframe_api_benchmark.py store_sales_1_4.$s.dat
  > bench_$$s.log 2>&1; done' > /dev/null 2>&1 &
$ for s in 100 200 400 800; do echo "==" $$s"; grep -E "Read input|iteration 1|iteration 5|Error|Exception" bench_$$s.log; done
== 100
INFO:root:Read input files : elapsed seconds: 13.6563
INFO:root:Query q1 (iteration 1): elapsed seconds: 40.6506
INFO:root:Query q2 (iteration 1): elapsed seconds: 34.6612
INFO:root:Query q3 (iteration 1): elapsed seconds: 2.6527

```

```

INFO:root:Query q4 (iteration 1): elapsed seconds: 2.5076
INFO:root:Query 5 (iteration 1): elapsed seconds: 13.686
INFO:root:Query q1 (iteration 5): elapsed seconds: 0.587
INFO:root:Query q2 (iteration 5): elapsed seconds: 1.2445
INFO:root:Query q3 (iteration 5): elapsed seconds: 1.2557
INFO:root:Query q4 (iteration 5): elapsed seconds: 1.4948
INFO:root:Query 5 (iteration 5): elapsed seconds: 11.3795
== 200
INFO:root:Read input files : elapsed seconds: 17.4995
INFO:root:Query q1 (iteration 1): elapsed seconds: 39.4248
INFO:root:Query q2 (iteration 1): elapsed seconds: 60.3215
INFO:root:Query q3 (iteration 1): elapsed seconds: 4.6922
INFO:root:Query q4 (iteration 1): elapsed seconds: 4.0913
INFO:root:Query 5 (iteration 1): elapsed seconds: 18.5396
INFO:root:Query q1 (iteration 5): elapsed seconds: 0.6943
INFO:root:Query q2 (iteration 5): elapsed seconds: 2.0858
INFO:root:Query q3 (iteration 5): elapsed seconds: 2.7961
INFO:root:Query q4 (iteration 5): elapsed seconds: 3.4506
INFO:root:Query 5 (iteration 5): elapsed seconds: 17.0232
== 400
INFO:root:Read input files : elapsed seconds: 25.1216
INFO:root:Query q1 (iteration 1): elapsed seconds: 40.8925
INFO:root:Query q2 (iteration 1): elapsed seconds: 119.3505
INFO:root:Query q3 (iteration 1): elapsed seconds: 7.8951
INFO:root:Query q4 (iteration 1): elapsed seconds: 6.7831
INFO:root:Query 5 (iteration 1): elapsed seconds: 28.1306
INFO:root:Query q1 (iteration 5): elapsed seconds: 1.0178
INFO:root:Query q2 (iteration 5): elapsed seconds: 3.42
INFO:root:Query q3 (iteration 5): elapsed seconds: 6.2839
INFO:root:Query q4 (iteration 5): elapsed seconds: 6.8643
INFO:root:Query 5 (iteration 5): elapsed seconds: 27.6679
== 800
INFO:root:Read input files : elapsed seconds: 42.0472
INFO:root:Query q1 (iteration 1): elapsed seconds: 40.6946
INFO:root:Query q2 (iteration 1): elapsed seconds: 283.9802
INFO:root:Query q3 (iteration 1): elapsed seconds: 42.332
INFO:root:Query q4 (iteration 1): elapsed seconds: 20.0374
INFO:root:Query 5 (iteration 1): elapsed seconds: 60.7687
INFO:root:Query q1 (iteration 5): elapsed seconds: 6.8292
INFO:root:Query q2 (iteration 5): elapsed seconds: 7.9088
INFO:root:Query q3 (iteration 5): elapsed seconds: 15.5996
INFO:root:Query q4 (iteration 5): elapsed seconds: 13.0538
INFO:root:Query 5 (iteration 5): elapsed seconds: 48.1083

```

Aucune erreur dans les logs. Le tableau donne la dernière itération, et entre parenthèses la première pour Q1 et Q2 :

Fichier (taille)	Read (s)	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)	Q5 (s)
store_sales_1_4.100 (9,5 Go)	13,66	0,59 (40,7)	1,24 (34,7)	1,26	1,49	11,38
store_sales_1_4.200 (18,9 Go)	17,50	0,69 (39,4)	2,09 (60,3)	2,80	3,45	17,02
store_sales_1_4.400 (37,4 Go)	25,12	1,02 (40,9)	3,42 (119,4)	6,28	6,86	27,67
store_sales_1_4.800 (76,3 Go)	42,05	6,83 (40,7)	7,91 (284,0)	15,60	13,05	48,11

Tableau 5 – 5^e itération (1^{re} itération entre parenthèses).

4.3 Analyse

Q1 et Q2 à la première itération. `cache()` ne charge rien tout de suite : les données sont lues et mises en mémoire par la première action qui s'en sert. Q1 est la première action sur `customer`, elle lit donc les 8,3 Go du fichier et remplit le cache. C'est pour ça qu'elle prend toujours environ 40 s, quel que soit le fichier de ventes. Q2 joue le même rôle pour `store_sales`, et son temps suit la taille du fichier (35, 60, 119 puis 284 s). Ensuite, tout est lu depuis le cache. Le temps « Read » ne compte que l'inférence du schéma, pas la vraie lecture.

Écarts entre les requêtes. Ils nous semblent logiques. Q1 et Q2 se font en un passage, sans échange entre les machines : chaque partition donne un résultat partiel et on combine quelques valeurs. Q3 regroupe par client, il faut donc un shuffle, d'où un temps un peu plus long. Q4 trie en plus, mais sur le résultat de Q3, qui ne fait qu'une ligne par client : le tri coûte peu et Q4 reste proche de Q3. Q5 est de loin la plus lente (4 à 9 fois Q4) car elle joint deux grosses tables : les ventes et les 65 millions de clients doivent être redistribués selon la clé client.

Évolution avec la taille. Du premier au dernier fichier, la taille est multipliée par 8. Jusqu'à 37 Go, les temps augmentent moins vite que les données (Q2 fois 3, Q3 fois 5 pour 4 fois plus de données), à cause des coûts fixes et, pour Q5, de la table `customer` qui ne change pas. À 76 Go, ça se dégrade : Q3 passe de 6,3 à 15,6 s pour 2 fois plus de données, et même Q1, qui ne touche que `customer`, passe de 1 à 6,8 s. Les executors n'ont que 1 Go chacun, soit environ 15 Go pour le cluster : les données ne tiennent plus en mémoire, Spark en garde une partie sur les disques locaux et doit faire de la place. Le cache de `customer` a sans doute été en partie retiré, ce qui expliquerait Q1. Ça reste bien plus rapide que de relire le CSV, car même sur disque le cache est dans le format binaire de Spark.